

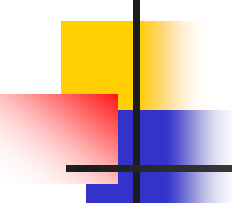


Introducción a la computación

SESION 06

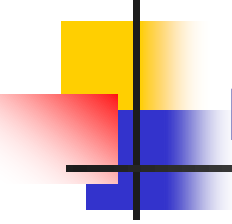
Introducción a la Algoritmia

UNIVERSIDAD NACIONAL DE INGENIERIA
Facultad de Ingeniería Industrial y de Sistemas
Ing. Miguel Angel Navarro Neyra
mnavarron@uni.edu.pe



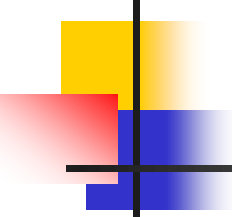
Agenda

- Memoria y operaciones aritméticas y lógicas
- Teorema de la programación estructurada
- Registro de instrucciones
- Elementos de un programa



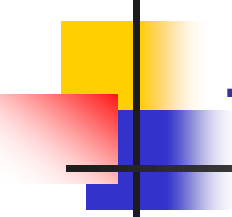
Memoria y Operaciones aritméticas y lógicas

- La memoria en un sistema informático es donde se almacenan datos e instrucciones. Existen diferentes tipos de memoria, como la RAM (memoria principal) y la memoria caché.
- Las operaciones aritméticas incluyen suma, resta, multiplicación y división. Estas operaciones se realizan en la Unidad Aritmético-Lógica (ALU) del procesador.
- Las operaciones lógicas (como AND, OR, NOT) se utilizan para evaluar condiciones y tomar decisiones en programas.



Agenda

- Memoria y operaciones aritméticas y lógicas
- Teorema de la programación estructurada
- Registro de instrucciones
- Elementos de un programa

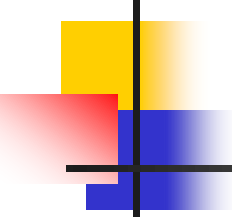


Teorema de la programación estructurada

Definición

El Teorema del programa estructurado es un resultado fundamental en la teoría de lenguajes de programación. Establece que toda función computable puede ser implementada en un lenguaje de programación utilizando solo tres estructuras de control:

- **Secuencia:** Ejecución de una instrucción tras otra.
- **Selección:** Ejecución de una de dos instrucciones (o conjuntos) según el valor de una variable booleana.
- **Iteración:** Ejecución de una instrucción (o conjunto) mientras una variable booleana sea 'verdadera'. Esta estructura también se conoce como ciclo o bucle.

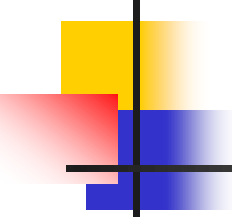


Teorema de la programación estructurada

Secuencia

Problema: Calcular el área de un triángulo:

- Pedir al usuario que ingrese la base del triángulo.
- Pedir al usuario que ingrese la altura del triángulo.
- Calcular el área utilizando la fórmula:
$$\text{Área} = (\text{base} * \text{altura}) / 2.$$
- Mostrar el resultado.

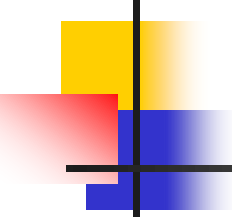


Teorema de la programación estructurada

Selección

Días de un mes

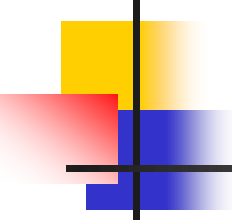
- Enero, marzo, mayo, julio, agosto, octubre o diciembre = 31 días
- Abril, junio, septiembre o noviembre = 30 días
- Febrero = 28 o 29 días



Teorema de la programación estructurada

Selección

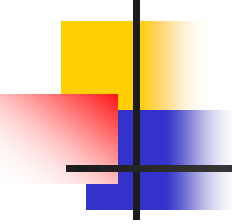
- Bifurcaciones **SI**
- Selecciones **SI**
- Decisiones **SI**
- Alternativas **SI NO**
- Condicionales **SI NO SI**



Teorema de la programación estructurada

Selección

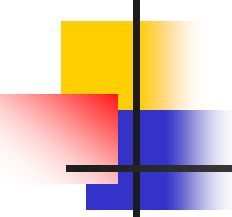
- IF
- ELSIF
- ELSE



Teorema de la programación estructurada

Selección

Si temperatura actual menor
que temperatura objetivo **entonces**
activar calefacción.

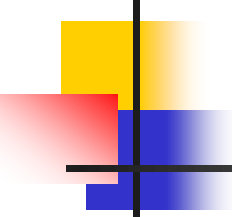


Teorema de la programación estructurada

Selección

Si temperatura actual menor
que temperatura objetivo **entonces**
activar calefacción.

En otro caso, para calefacción.

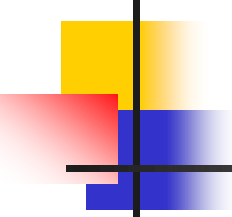


Teorema de la programación estructurada

Selección

Local

- Menores de 12 años no pueden entrar
- Entre 12 y 18 necesitan autorización
- Mayores de 18 años pueden entrar



Teorema de la programación estructurada

Selección

Constante edad mínima acceso = 12

Constante edad mínima acceso libre = 18

Variable edad cliente

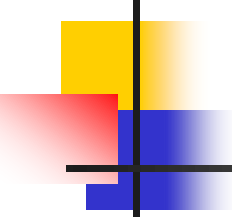
Si edad cliente < edad mínima acceso, entonces acceso denegado

Si no, si edad cliente < edad mínima acceso libre, entonces

Si presenta autorización entonces acceso permitido

Si no (si no presenta autorización) entonces acceso denegado

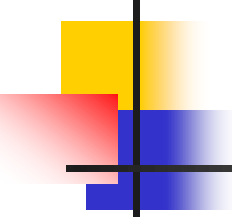
En otro caso (edad cliente mayor que edad mínima acceso libre)
entonces acceso permitido



Teorema de la programación estructurada

Iteración / Repetición

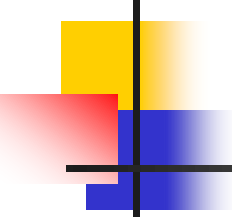
- Repetición de una misma acción
- Dinero en la cartera = suma de billetes y monedas
- Encontrar tijeras = abrir cajones



Teorema de la programación estructurada

Iteración / Repetición

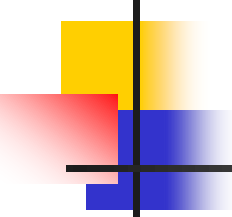
- Repetición de una misma acción
- Dinero en la cartera = suma de billetes y monedas
- Encontrar tijeras = abrir cajones



Teorema de la programación estructurada

Iteración / Repetición

Repetir una acción hasta conseguir el fin

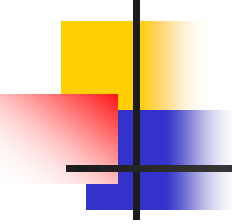


Teorema de la programación estructurada

Iteración / Repetición

Tipos de bucles

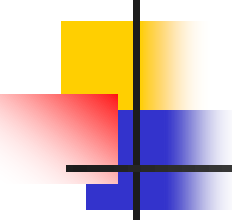
- Para todo $\forall \rightarrow$ **for**
 - Recorrer una lista de elementos hasta el final
 - Repetir n veces la misma acción
- Existe $\exists \rightarrow$ **while**
 - Parar repetición al encontrar



Teorema de la programación estructurada

Iteración / Repetición

¿for o while?

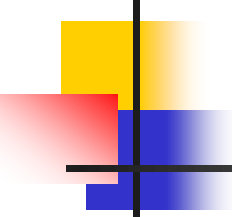


Teorema de la programación estructurada

Iteración / Repetición

Pasar lista en clase

- ¿for o while?



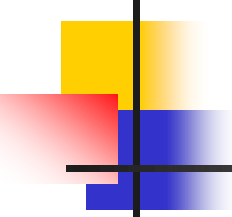
Teorema de la programación estructurada

Iteración / Repetición

¿Cuál es la capital de Francia?

- ¿for o while?

~~1, 2, 3, 4, 5, ...~~ 40



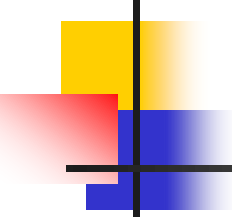
Teorema de la programación estructurada

Iteración / Repetición

¿Cuál es la capital de Francia?

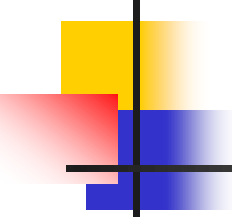
- ¿for o while?

1 → 2 → 3 → 4



Agenda

- Memoria y operaciones aritméticas y lógicas
- Teorema de la programación estructurada
- Registro de instrucciones
- Elementos de un programa

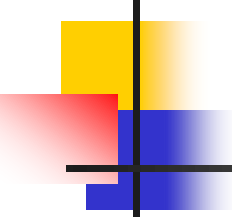


Registro de instrucciones



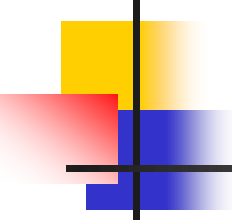
TEMA 1: El ordenador y la información

Fundamentos de Informática
(Grados en Ingenierías Industriales)



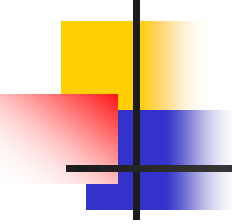
Agenda

- Memoria y operaciones aritméticas y lógicas
- Teorema de la programación estructurada
- Registro de instrucciones
- Elementos de un programa



Elementos de un programa

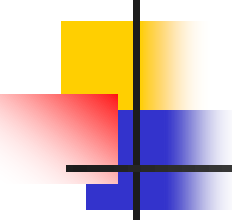
- Identificadores
- Variables y valores
- Tipos de datos primitivos
- Literales
- Operadores



Elementos de un programa

Identificadores

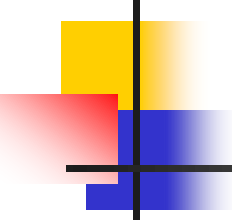
- Los identificadores son nombres que los programadores asignan a variables, funciones, objetos y otros elementos en el código.
- Pueden contener letras (mayúsculas y minúsculas), números y guiones bajos (_), pero no pueden comenzar con un número.
- Ejemplos válidos de identificadores son "miVariable" o "contador_1".



Elementos de un programa

Variables y valores

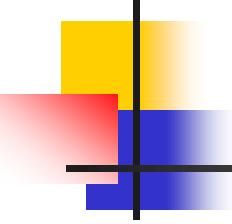
- Las variables son espacios de memoria donde se almacenan datos.
- Cada variable tiene un nombre y un valor asociado.
- Los valores pueden ser números, texto, fechas, etc.



Elementos de un programa

Tipos de datos primitivos

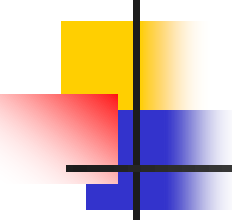
- Los tipos de datos primitivos son valores simples utilizados en programación.
- Algunos ejemplos son:
 - Enteros: Representan números enteros (por ejemplo, 42).
 - Decimales: Almacenan números con parte decimal (por ejemplo, 3.14).
 - Cadenas de texto: Contienen secuencias de caracteres (por ejemplo, "Hola").
 - Valores lógicos: Representan verdadero (true) o falso (false).



Elementos de un programa

Literales

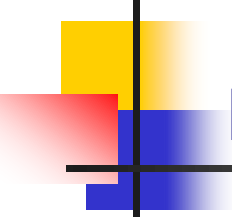
- Los literales son valores constantes utilizados directamente en el código.
- Pueden ser numéricos (como 10), de texto (como "Hola") o de caracteres especiales (como '\n' para salto de línea).



Elementos de un programa

Operadores

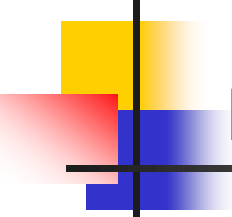
- Los operadores realizan operaciones en el código.
- Algunos tipos de operadores son:
- Aritméticos: Suma, resta, multiplicación, división, etc.
- Relacionales: Comparan valores (por ejemplo, ==, >, <).
- Lógicos: Combinan valores booleanos (por ejemplo, &&, ||).



Memoria y Operaciones aritméticas y lógicas

Operadores: Aritméticos

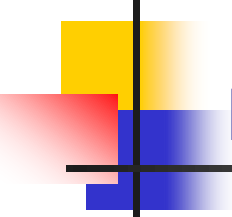
OPERADOR	DEFINICIÓN	EJEMPLO	
-	Restar números	Datos: $A=6-3$	Resultado: $A=3$
+	Sumar números	Datos: $A=6+3$	Resultado: $A=9$
*	Multiplicar números	Datos: $A=6*3$	Resultado: $A=18$
/	Dividir números	Datos: $A=6/3$	Resultado: $A=2$
^	Elevar a una potencia un numero	Datos: $A=6^3$	Resultado: $A=216$
--	Quitar una unidad a un numero	Datos: $A=6$; $A=--$	Resultado: $A=5$
++	Aumentar una unidad a un numero	Datos: $A=6$; $A=++$	Resultado: $A=7$
%	Obtener el modulo de una división, es decir, el residuo de dicha división	Datos: $A=6$, $B=4$; $A=A\&B$	Resultado: $A=2$



Memoria y Operaciones aritméticas y lógicas

Operadores: Relacionales

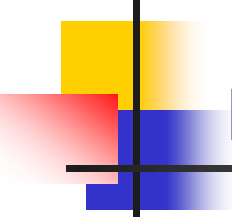
OPERADOR	DEFINICIÓN	EJEMPLO
<	Menor que	Datos: A=6, B=3 Comparación: If(A<B) Resu=F
>	Mayor que	Datos: A=6, B=3 Comparación: If(A>B) Resu=V
<=	Menor o igual que	Datos: A=6, B=3 Comparación: If(A<=B) Resu=F
>=	Mayor o igual que	Datos: A=6, B=6 Comparación: If(A>=B) Resu=V
<> ó !=	Diferente a	Datos: A=6, B=3 Comparación: If(A!=B) Resu=V
=	Igual que	Datos: A=6, B=3 Comparación: If(A=B) Resu=F



Memoria y Operaciones aritméticas y lógicas

Operadores: lógicos

OPERADOR	DEFINICIÓN	EJEMPLO
&& AND	Al comparar valores o expresiones, si ambos son verdaderos obtiene un "true"	Datos: A=6; B=6 Comparación: If(A>3 && B>4) Si se cumple "Cierto", si no "Falso"
OR	al comparar valores o expresiones, si una es verdadera obtiene un "true", si ambas son falsas se obtiene un "false"	Datos: A=6; B=6 Comparación: If(A>3 B>22) Si se cumple "Cierto", si no "Falso"
!= NOT	Al comparar valores o expresiones, niega la respuesta o resultado obtenido.	Datos: A=6; B=6 Comparación: If(A>3 != B>22) Si se cumple "Cierto", si no "Falso"

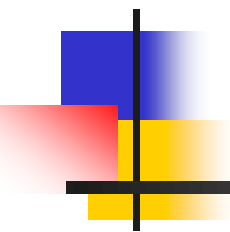


Memoria y Operaciones aritméticas y lógicas

Operadores: ***de asignación***

OPERADOR	DEFINICIÓN	EJEMPLO
+=	la suma del valor de la izquierda más el de la derecha	Datos: A=6; B=2, Expresión: A+=B Resultado: A=6+2
-=	la resta del valor de la izquierda menos el de la derecha	Datos: A=6; B=2, Expresión: A-=B Resultado: A=6-2
=	la multiplicación del valor de la izquierda por el de la derecha	Datos: A=6; B=2, Expresión: A=B Resultado: A=6*2

Introducción a la computación



FIN SESION 06